

1. A Failed Check Is Information

When a verification step fails, the wrong response is to treat every failure as proof that the latest edit is wrong. The check may expose the target bug, a regression caused by the patch, an unrelated pre-existing failure, flaky infrastructure, or an incorrect test assumption.

The next action should discriminate among these possibilities.

2. Failure Classification

Ask four questions:

1. Did this exact check fail before the change?
2. Is the failure in code touched by the change or in a dependent path?
3. Is the result reproducible under the same conditions?
4. What observation would distinguish a code defect from an environment or test defect?

A baseline is valuable because it separates "the repository is red" from "my patch made the repository red." When the baseline cannot be run, state that limitation rather than pretending the causal question was answered.

3. The Patch-to-Test Trap

Suppose Codex changes a function and writes a test that passes. That pair can still be jointly wrong. The test may mirror the new implementation instead of the intended contract. A stronger response is to derive at least one check from an external source of truth: a specification, previous behavior, user reproduction, protocol requirement, or independent invariant.

This matters especially when Astra does product thinking. A coherent implementation can invent a coherent behavior that was never requested.

4. Review as a Different Lens

Codex code review can inspect a diff in a read-only mode and prioritize findings without modifying the worktree. Review is useful after implementation because its objective changes: not "finish the task" but "find reasons the proposed change should not be accepted."

The roles can be separated even within one model. The implementer follows the success path. The reviewer searches for counterexamples, regressions, security implications, missing tests, and mismatches between stated intent and diff.

5. Adversarial Verification

For consequential changes, ask a reviewer to falsify the completion claim. Examples:

- Find an input that violates the new invariant.
- Compare the diff with the original issue and list requirements not actually tested.
- Trace call sites that may rely on previous behavior.
- Inspect error paths, not only the happy path.
- For UI, attempt the flow at narrow width, keyboard-only, and slow network states.

A review that cannot find an issue is not proof of correctness. It is evidence whose value depends on the search performed.

6. Flakiness

If a test fails once and passes on rerun, do not automatically classify it as irrelevant. Record the instability. Ask whether the change altered timing, ordering, shared state, or external calls. A flaky signal may be weaker than a deterministic one, but repeated disappearance can itself be evidence of a race or hidden dependency.

7. The Correct Final Report

A trustworthy handback distinguishes:

Changed: what was modified.

Verified: checks actually run and their results.

Not verified: relevant checks that could not be run.

Observed uncertainty: failures or assumptions still open.

Review state: whether an independent read-only review occurred and what it found.

Do not replace this with "all good". Evidence is more useful than confidence language.

References: OpenAI, Code review in Codex; OpenAI, GPT-6 Astra model guidance; OpenAI, Codex use cases.

Independent Open Course

<https://github.com/dicnunz/codex-astra-course>

CODEX.00 | Codex with GPT-6 Astra

Handout 9: Responding to Verification Failures

Fall 2026

License and source

Structural and pedagogical adaptation of MIT OpenCourseWare 24.00 Problems of Philosophy, Fall 2019, Prof. Miriam Schoenfield.

Original course and this adaptation are licensed CC BY-NC-SA 4.0.

This course is independently produced and is not affiliated with or endorsed by MIT or OpenAI.

Linked external readings and referenced product documentation retain their own copyrights and terms.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

<https://ocw.mit.edu/courses/24-00-problems-of-philosophy-fall-2019/>